CONF42 PLATFORM ENGINEERING 2026

Platform SLOs Beyond Uptime

Measuring the journey from request to value



Amith Reddy Ravuru
Staff Platform Engineer · Reltio, An SAP Company

THE PROBLEM

All green, still waiting

PLATFORM DASHBOARD · THIS WEEK

API availability	p99 latency	Error rate	Cluster CPU
99.95%	180 ms	0.02%	41%
SLO met	SLO met	SLO met	Within limits

ONE DEVELOPER'S REQUEST · SAME WEEK

Mon 09:10	Mon 15:40	Tue 11:00	Wed 16:30	Fri 10:20
Requests a new tenant	Quota error, opens a ticket	Chat thread for network access	Waits on a policy exception	**First deploy works**

Every SLO met. Four days to a working environment.

Illustrative example. Numbers and timeline show the pattern, not real data.

WHY IT HAPPENS

Where operational SLOs stop

WHAT OUR SLOS COVER

Request portal
SLO met

queue

Workflow engine
SLO met

approval

Cloud APIs
SLO met

manual fix

Policy engine
SLO met

SAME REQUEST, LAID OUT IN TIME · GREEN WIDENED SO YOU CAN SEE IT



Services doing work: about 40 minutes Waiting on tickets, approvals and people: about 4 days

Green on the outside, red on the inside. The watermelon effect.

Illustrative numbers · Sources: HappySignals; Google SRE book, ch. 4

THE MODEL

The journey, request to ready

01
Requested
Asks via portal, CLI or a pull request.

02
Provisioned
Account, subscription or project exists.

03
Usable
Logs in. First deploy works.

04
Policy-compliant
Guardrails pass. No open exceptions.

05
Operationally ready
Monitored, alerting, backed up, cost-tagged.

CARRIED END TO END

```
journey_id = tnt-0142 · minted at request, carried to ready
```

THE MODEL

What a journey SLO is

```
Journey SLI = good journeys ÷ all journeys in the window
```

	Operational SLO	Journey SLO
Unit	One API request	One developer journey, end to end
Good means	Succeeded, under 300 ms	Reached ready in time, no rework
Data from	Service metrics	Workflow events, tickets, a pulse survey
Owned by	Each service team	The platform team, as owner of the path
Answers	Is the service up?	Did they get what they came for?

Same ratio form as SRE SLIs: good events ÷ total events (Google SRE workbook)

THE INDICATORS

Six journey indicators

Faster	**Time to first environment** Request to first working deploy	**Time to operational readiness** Request to ready for real traffic
Safer	**Failed-workflow recovery time** Failed run to developer moving again	**Rework rate** Journeys where something was redone
Easier	**Self-service completion rate** Journeys finished with no human help	**Support dependency** Help requests per 100 journeys

Start with at least one from each row.

FASTER · INDICATOR 1 OF 6

Time to first environment

From the request to the developer's first working deploy on the new environment.

HOW TO MEASURE

Start: request created, journey ID minted

Stop: first successful deploy by the requester

Report: p50 and p90, never the average

EXAMPLE TARGET

4 h

90% of journeys get a working deploy within 4 hours

Illustrative target. Set yours from a 2 to 4 week baseline of your own journeys.

FASTER · INDICATOR 2 OF 6

Time to operational readiness

From the request to an environment you could put real traffic on.

HOW TO MEASURE

Start: request created, same journey ID

Stop: the last readiness check passes: monitors, alert routing, backups, cost tags, no open exceptions

Report: p50 and p90, plus which check finished last

EXAMPLE TARGET

2 days

90% of journeys fully ready within 2 working days

Illustrative target. Write your own ready checklist down, then automate it.

SAFER · INDICATOR 3 OF 6

Failed-workflow recovery time

When a provisioning run fails, the time until the developer is moving again.

HOW TO MEASURE

Start: a workflow step fails for a journey

Stop: that step succeeds for the same journey, by retry or by a fix

Report: p90, split by cause: quota, permissions, policy, cloud API, our bug

EXAMPLE TARGET

1 h

90% of failed runs recovered within 1 hour

Illustrative target. Same idea as DORA's failed deployment recovery time, applied to workflows.

SAFER · INDICATOR 4 OF 6

Rework rate

The share of journeys where something had to be redone after we said done.

HOW TO MEASURE

Count: re-requests, manual fixes, teardown and rebuild, policy fixes after handoff
Window: within 14 days of the journey finishing
Report: % of finished journeys with any rework

EXAMPLE TARGET

< 10%

Fewer than 1 in 10 journeys need rework

Illustrative target and window. Related in spirit to DORA's deployment rework rate, but not the same metric.

EASIER · INDICATOR 5 OF 6

Self-service completion rate

The share of journeys that reach usable with no human stepping in.

HOW TO MEASURE

Count: journeys that reached usable this week

Good: no manual step, no ticket someone had to action, no side-channel approval

Report: % of journeys, split by path and by cloud

EXAMPLE TARGET

85%

85% or more of journeys finish with no human stepping in

Illustrative target. A human step can be fine. Hiding one behind a button is not.

EASIER · INDICATOR 6 OF 6

Support dependency

How often developers had to ask a person for help to get through.

HOW TO MEASURE

Count: tickets, chat threads and DMs tied to a journey
Normalize: per 100 journeys, so a busy month doesn't look like a bad one
Report: by journey stage and by topic

EXAMPLE TARGET

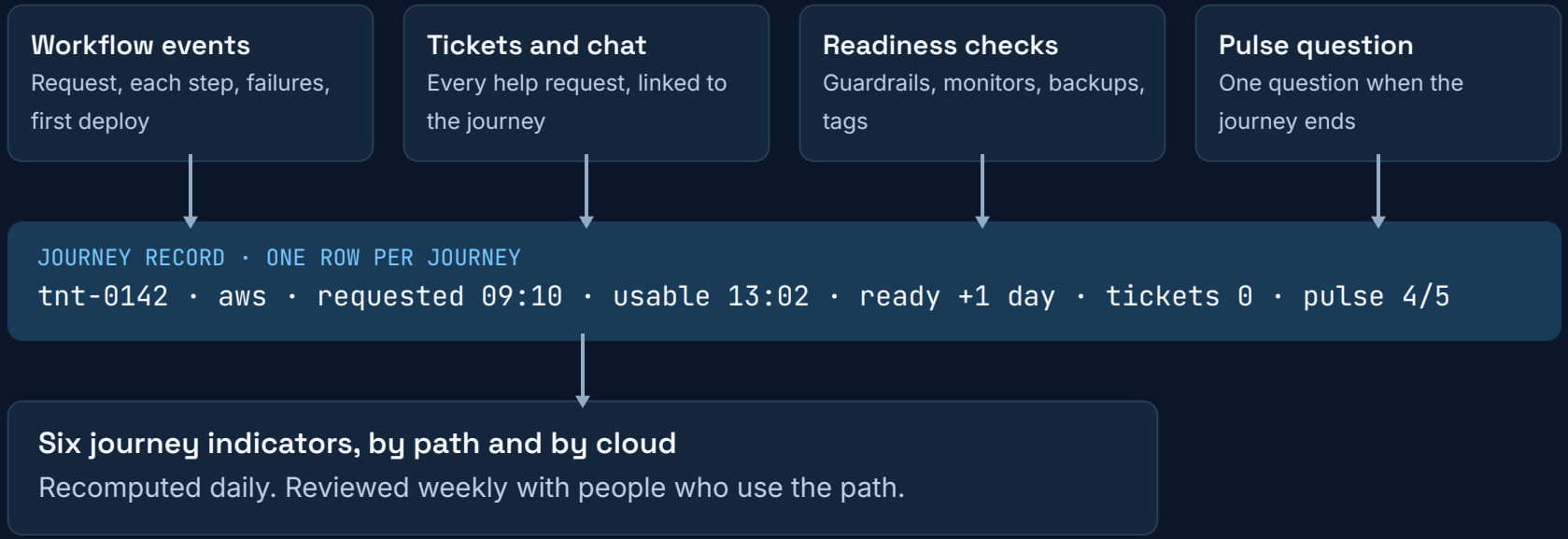
< 15

Under 15 help requests per 100 journeys

Illustrative target. Questions count too: confusion is cognitive load (DevEx, ACM Queue 2023).

MAKING IT REAL

One journey ID joins the data



Why both kinds of data: DevEx: What Actually Drives Productivity, ACM Queue, 2023

EXAMPLE

One tenant, many component owners

01

Ordered

An order arrives with its bill of materials.

STARTS HERE

Journey ID minted. Every clock starts.

02

Planned

The order and target cloud become a component plan.

MEASURED HERE

- Plan resolution time
- Plan first-pass accuracy

03

Provisioned

Each owner provisions its part, in parallel.

MEASURED HERE

- Handoff wait per owner
- Component time vs target
- Critical-path component

04–06

Usable to ready

The same finish line as the rest of the talk.

MEASURED HERE

- BOM conformance
- The six indicators

JOURNEY RECORD · THIS TENANT

`tnt-0217 · cloud B · 7 components · slowest: search · flow efficiency 18% · BOM exact`

Illustrative example. Components, clouds and numbers are placeholders.

USING IT

Six ways this goes wrong

Anti-pattern	Do this instead
Reporting the average	Report p50 and p90. One stuck journey vanishes in an average.
Stopping the clock at provisioned	Stop it at ready, as the requester sees it.
Pushing one indicator on its own	Pair it: speed with rework, self-service with support.
A human step behind a button	Log every manual action against the journey ID.
Targets before a baseline	Measure 2 to 4 weeks first. Then pick a number you can defend.
Ranking teams by their numbers	Use the record to find the slowest step, then fix it with that owner.

Each one makes the dashboard look better without making the journey any faster.

USING IT · THE FRAMEWORK

Your own journey SLOs in 30 days

Week 1

Define done

Pick one journey that hurts.

Write down done, from the requester's side.

Map its stages, request to ready.

Week 2

Instrument

Mint a journey ID at the request.

Carry it on events, tickets and resources.

Pick one indicator from each pair.

Week 3

Baseline

Let real journeys run through.

Read the records. Find the slowest stage.

Hold off on targets.

Week 4

Commit

Set targets from the baseline.

Agree a journey error budget.

Review weekly with people who use the path.

KEEP AN INDICATOR ONLY IF

The requester would feel it · You can compute it per journey · Someone owns fixing it

WRAP-UP

Three things to take home

01

Measure the whole journey

Service SLOs cover the minutes. Developers live in the days between them.

02

Stop the clock at ready

Pair every speed indicator with a quality one, so neither can be gamed.

03

Join it with one ID

Mint it at the request. Most of the data already exists.

Start the clock at the request. What does your golden path really take?